

Stable Diffusion

guide for use

Lahinthe futureland

Text2img

- Nội dung tài liệu được tham khảo từ nhiều nguồn cả Việt Nam và thế giới cũng như chiêm nghiệm cá nhân trong quá trình làm việc với SD.
- Bài viết sẽ rất nhiều chữ, vì thực ra mình là dân làm kỹ thuật. Bệnh nghề nghiệp là nếu viết ít thì sợ ko đủ ý, và ko giải thích được đầy đủ.
- Mình cũng rất cố gắng viết 1 cách ngắn gọn nhất để mọi người có thể dễ tiếp cận nhất có thể.
- Vì tài liệu mình share công khai nên mong các bạn đừng xóa watermark của mình nhé, nó cũng bé xíu thôi à!

Table of content

Part 1: Text2img

- Prompts/Negative prompts
- Sampling method/Sampling steps
- Batch count/Batch size/Seed
- Base model
- Lora/Lycoris/Loha
- Something other
- Workflow

Lahin the futureland

Texting

Prompts/Negative prompts

Lahin the futureland

Texting Prompts

Prompts hay lời nhắc là thứ mà hầu như bạn sẽ luôn phải nhập khi sử dụng các mô hình AI hiện tại.

Về cơ bản Prompt là đầu vào ở đây là dưới dạng text bao gồm 1 hoặc nhiều câu, từ miêu tả về đầu ra bạn mong muốn.

Ví dụ:

1dog is standing on the street



Bạn có thể thêm các thành phần để miêu tả rõ hơn về chủ thể như màu sắc, kiểu dáng, các yếu tố xung quanh.

Ví dụ:

1 dog is standing on the street, white dog, sky, cloud, colorful background



Texting Prompts

Lahin the futureland

Vậy cấu trúc của 1 prompt như thế nào thì hiệu quả?
Mình hay sử dụng công thức dưới đây:

Quality + subject + accessory + environment + artstyle

- Quality đơn giản là chất lượng, bạn có thể copy từ rất nhiều các prompt khác những key như: Masterpiece, high resolution, 4k, 8K, 64K, best quality...
- Subject là chủ thể nhắc tới, có thể là 1 câu cũng có thể là 1 từ. Ví dụ muốn miêu tả 1 cô gái đang nằm trên giường, có thể viết 2 cách: 1girl, lying on the bed – hoặc 1girl is lying on the bed. Đừng quá lo lắng về ngữ pháp vì thực ra là AI có thể hiểu điều bạn muốn nói.
- Accessory. Ý như mặt chữ. Ví dụ cô gái đeo vòng cổ, mặc áo thun, và chân váy chúng ta có prompt: necklace, wearing T-shirt, wearing skirt. Nhớ là ko cần thiết phải viết thành 1 câu hoàn chỉnh, có nghĩa. Chỉ cần các cụm từ cách nhau bởi dấu (,) miêu tả rõ yêu cầu là được
- Enviroment: môi trường xung quanh, đại khái là miêu tả không gian xung quanh chủ thể. Trong nhà hay ngoài trời, địa điểm ở đâu? Ví dụ: outdoor, on the street, night city, moon light...
- Art style: Trước đây SD cho phép bạn có thể gõ tên các artist trên artstation và tự dung sẽ có 1 bức tranh với nét vẽ của artist đó. Tuy nhiên thì biểu tình dữ dội quá cho nên SD đã bỏ tính năng này. Chủ yếu là bạn xác định lại phong cách hình ảnh của mình là realistic, anime, 2D, 3D, fantasy... Thích cái gì thì điền vào cái đấy

Testing Prompts

Có nhiều ý kiến cho rằng viết prompt càng chi tiết thì càng tốt, nhưng thực ra ko phải như thế. Prompt càng dài AI càng khó thực hiện. Vì thế hãy đảm bảo có đủ 5 thành phần cấu thành nên Prompt, và viết nó ngắn gọn xúc tích nhất có thể.

Để hiểu rõ hơn thì có thể tìm bài của bác Anh Hoàng đã chia sẻ về lệnh Break trong prompt. Ở đây mình có thể tóm tắt là, thực tế 1 prompts trong SD sẽ tối ưu khi giới hạn ở 75 ký tự (1 chunk). Sau 75 ký tự sẽ bắt đầu lại 1 chunk mới. Nếu prompt của bạn nằm giữa 2 chunk này thì AI sẽ tự tách ra thành 2 lệnh khác nhau

Chunk 1

A dog standing on

Chunk 2

a green car

Thay vì vẽ 1 con chó đứng trên ô tô thì SD đã hiểu câu prompt là: 1 con chó đang đứng, cái xe ô tô màu xanh

LahintheFutureland



Testing Negative prompts

Nếu bạn cảm thấy mình đã viết prompts chi tiết, xúc tích lắm rồi mà con AI nó vẫn ngu thì có thể tìm đến Negative prompts.

Đối lập với prompt, bạn ko thích cái gì xuất hiện thì điền vào chỗ này. Thực ra negative prompts trong SD rất quan trọng, vì lỗi quái vật 3 đầu 6 tay thường xuyên xuất hiện. Cho nên 1 số key gần như lúc nào cũng có trong negprompt đại loại là: extra hand, extra leg, missing hand, missing leg, missing finger, extra finger....

Tất nhiên ko phải lúc nào cũng thứ này cũng work, tuy nhiên nó sẽ hạn chế được phần nào các lỗi xảy ra.

Lahin the futureland



Texting

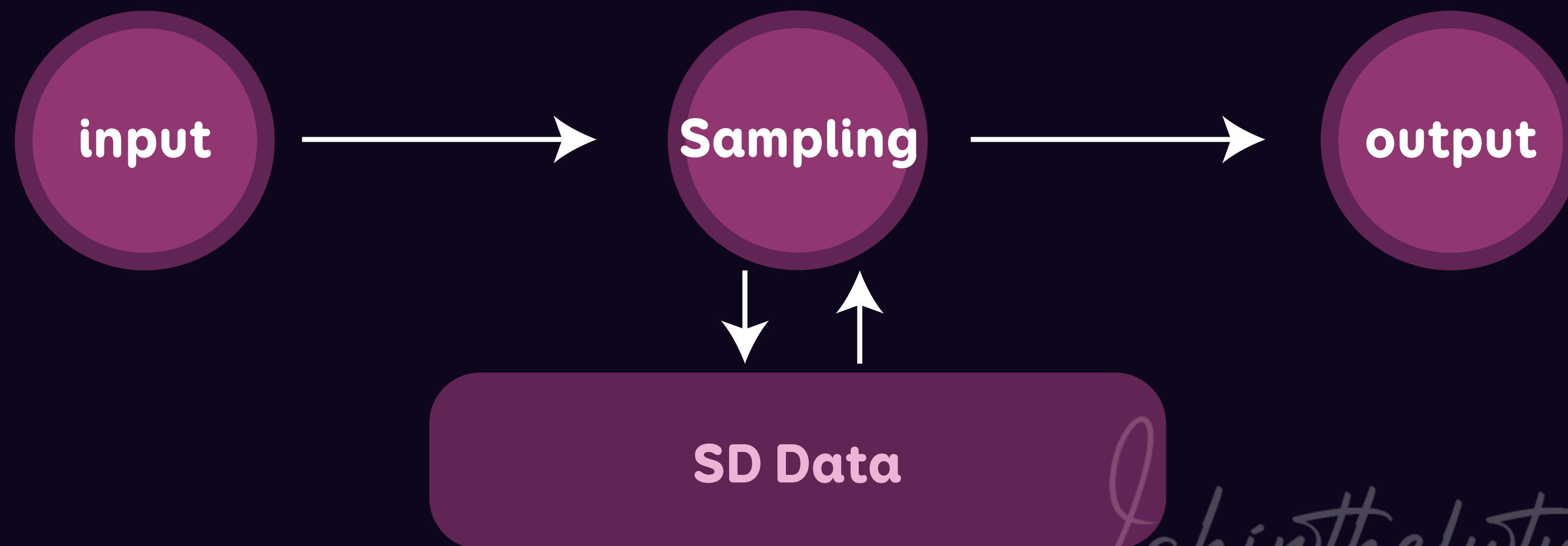
Sampling method/ Sampling steps

Lahinthe futureland

Texting

Sampling method/ Sampling steps

Sampling là quá trình lấy mẫu. Về cơ bản thì SD sẽ xử lý input của bạn (cụ thể ở đây là text), sau đó tìm kiếm những kết quả phù hợp nhất với input trong dataset của mình để cho ra output.



Lahin the futureland

Testing Sampling method/ Sampling steps

Sampling method

Hiện tại có rất nhiều cách thức lấy mẫu: Euler, DDIM, UniPC, DPM Karas, SDE... Có thể hiểu đơn giản là chúng ta có rất nhiều nhân viên cùng vẽ 1 bức tranh, nhưng workflow của mỗi ông mỗi khác. Có ông thì phóng khoáng đặt bút vẽ ngay từ đầu, về sau chỉ chỉnh sửa rất ít, cũng có ông bước đầu tập trung lên layout trước rồi về sau mới tạo hình, tô màu. Quá trình vẽ của các ông này đều được hiển thị khi các bạn generate hình ảnh ở dạng bị blur. Workflow cũng sẽ ảnh hưởng đến tốc độ làm việc của từng ông. Cho nên có ông vẽ nhanh, có ông vẽ chậm. 1 số ý kiến cho rằng DPM + SDE Karras sẽ luôn cho ra kết quả đẹp nhất. Cái này mình nghĩ còn tùy thuộc vào kỳ vọng đầu ra của mỗi người. Nhưng ông nào vẽ càng lâu thì cơ bản là đầu ra sẽ càng nhiều chi tiết.

Để giải thích chi tiết về từng phương pháp lấy mẫu thì mình hơi lười viết. Cho nên các bạn có thể tự tìm hiểu nhé, Hiện tại mình có thể suggest bạn sử dụng Euler a và UniPC nếu muốn xử lý tốc độ cao, và DPM+SDE Karras tốc độ chậm nhưng đầu ra sẽ chi tiết hơn.

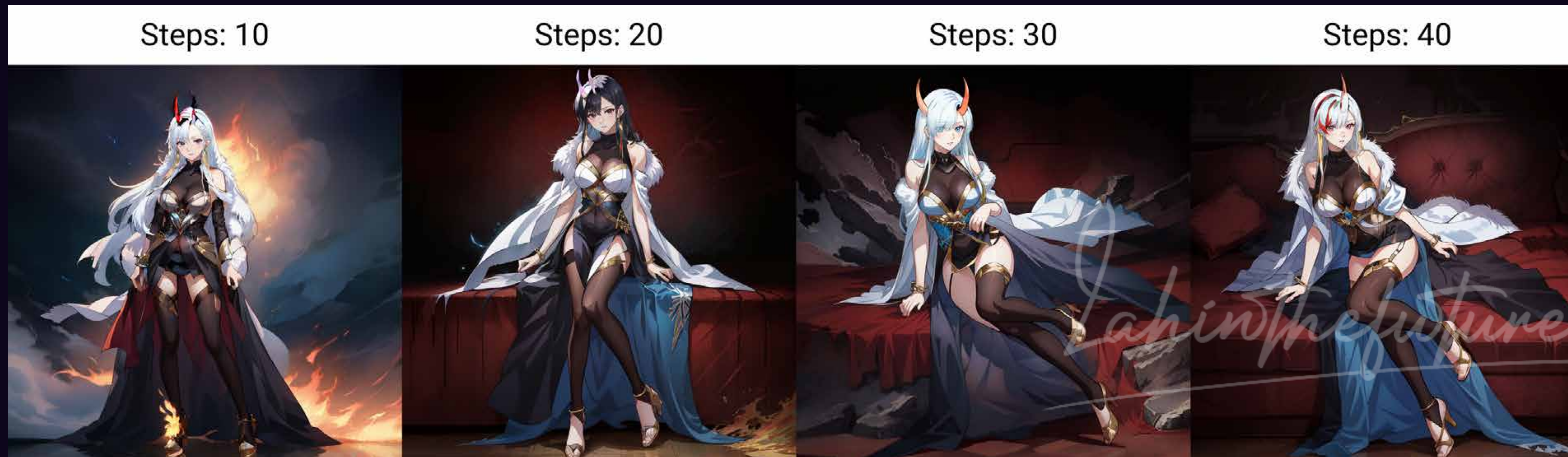
Euler a



DPM++ 2M SDE
Karras



Testing Sampling method/ Sampling steps



Sampling step

Số bước lấy mẫu. Vẫn là từng ấy nhân viên với các workflow khác nhau. Tuy ko thể can thiệp bắt đám nhân viên này thay đổi workflow (bạn có quyền đuổi việc nếu ghét), nhưng bạn có thể tăng độ khó công việc hoặc giảm thiểu áp lực cho đám nhân viên này bằng sampling step. Ví dụ là về bản chất workflow của 1 nhân viên được chia làm 3 giai đoạn, và sampling step mặc định là 20. Tức là mỗi giai đoạn trong workflow của nhân viên này sẽ được chia nhỏ làm các bước nhỏ hơn sao cho tổng cộng là 20 step. Nhưng nếu bạn muốn nhân viên của mình làm việc tỉ mỉ và chi tiết hơn, bạn có thể tăng số step lên nhiều hơn để nhận 1 kết quả tốt hơn. Tuy nhiên không phải lúc nào bắt nhân viên làm việc 1 cách rườm rà cũng cho 1 kết quả tốt hơn. Vì khi chia nhỏ đến 1 giai đoạn nhất định mà workflow của họ ko thay đổi thì dù có chia nhỏ hơn nữa cũng chả có gì để làm. Cơ bản thì mình suggest con số khoảng 20-50 cho hầu hết các trường hợp sử dụng.

Texting

Batch count, batch size, Seed

Lahinthe futureland

Testing

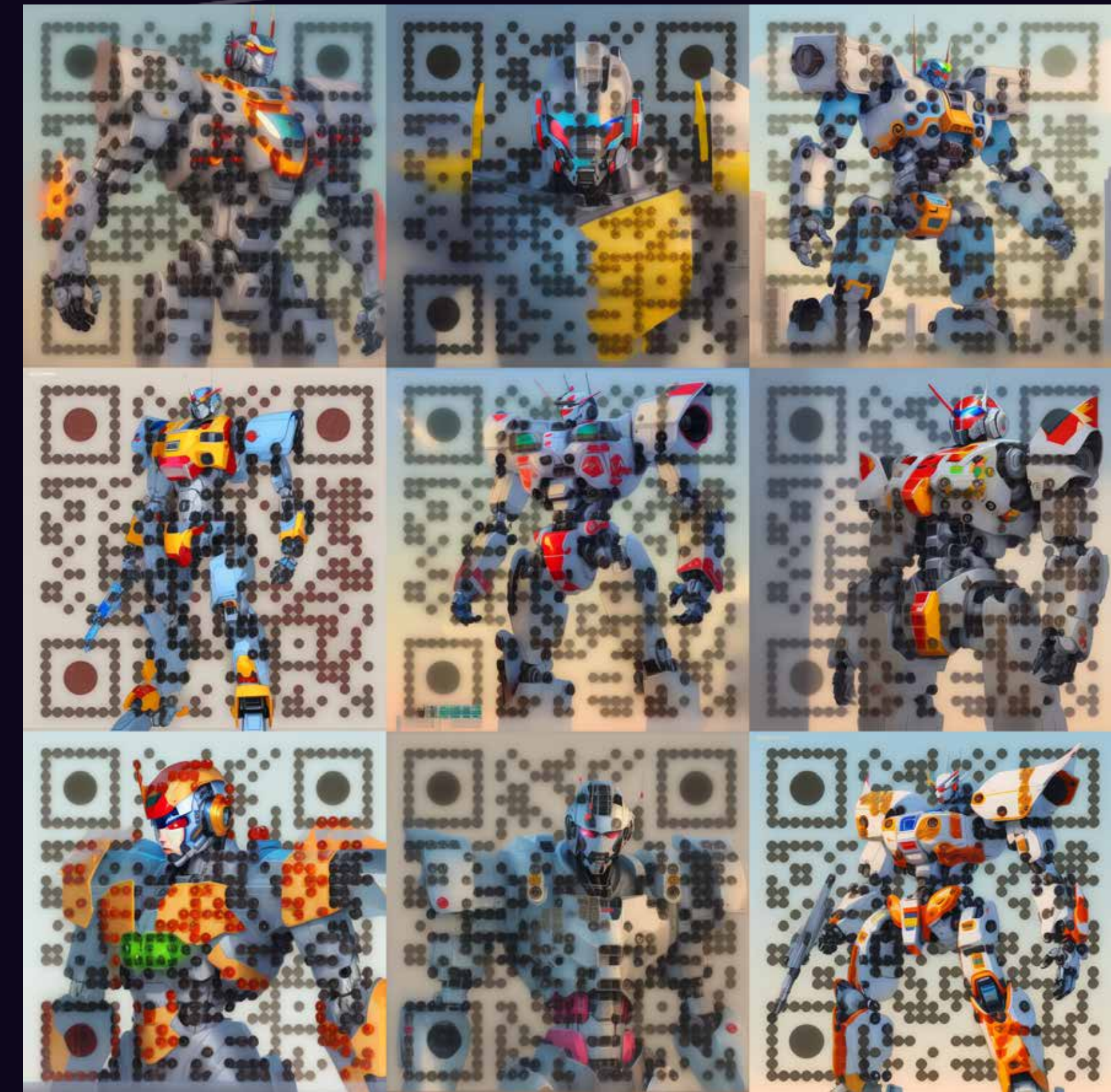
Batch count, batch size, Seed

Phần này khá đơn giản nên mình sẽ giải thích nhanh.

Batch count: là số kết quả đầu ra bạn muốn có trong 1 lượt generate. (thực ra mình rất ít dùng cái này vì SD có chức năng generate forever). Tuy nhiên nó cũng khá hữu ích trong 1 số trường hợp cần độ chính xác. Ví dụ bạn muốn tạo hình ảnh gì đó kèm với mã QR, thì có thể đặt batch count để kiểm tra tỉ lệ thành công khi tạo mã.

Batchsize: là số lượng hình ảnh bạn muốn xử lý trong cùng 1 thời điểm. Ví dụ batchsize = 2 thì SD sẽ cho ra 2 kết quả cùng lúc, tương tự với 3-4-5. Tuy nhiên điều ấy đồng nghĩa với việc chiếc VGA của bạn đang khóc trong góc máy tính.

Seed: cái này như là mã số của kết quả nhằm tạo tính đa dạng cho hình ảnh. Khi đặt seed = -1 thì mỗi lần tạo hình sẽ tự random 1 mã số khác nhau. Nếu bạn ko thay đổi gì về tất cả và sử dụng lại 1 giá trị seed cũ, bạn sẽ được 1 kết quả y chang kết quả trước đó.



Đặt batchcount = 9 để test có bao nhiêu hình có thể quét mã thành công

Lahin the futureland

Texting

Base model/ checkpoint

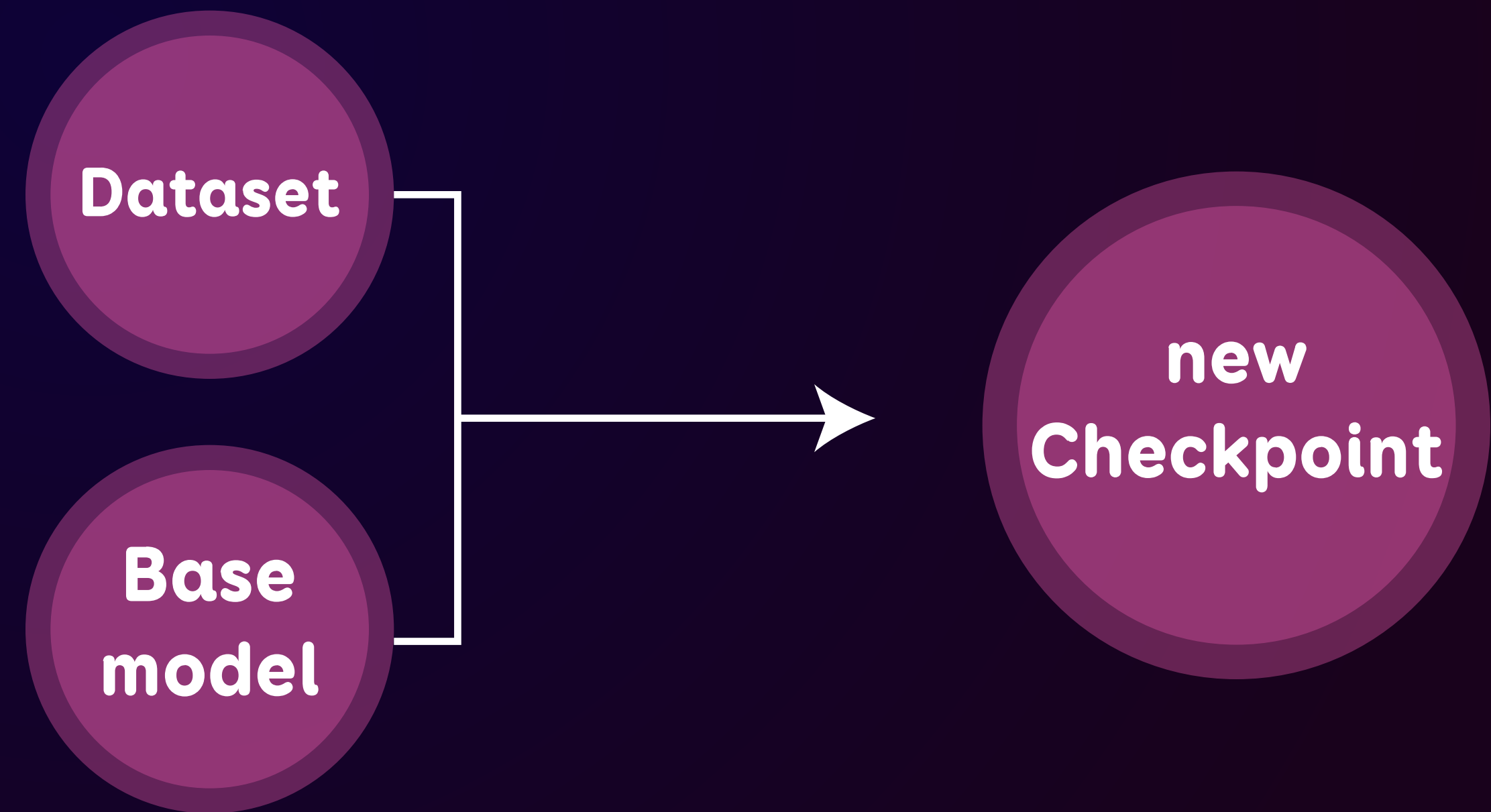
Lahin the futureland

Testing

Base model/ checkpoint

Lahin the futureland

- Nếu bạn muốn trở thành 1 generator AI artist thì phần này và phần ngay sau đây về lora là vô cùng quan trọng.
- Base model có thể hiểu là 1 ngăn chứa bao gồm các dữ liệu cơ bản của stable diffusion. Khi mới cài đặt và sử dụng SD bạn thường được hướng dẫn tải 1 file stablediffusionv1.5.cpkt hoặc đuôi safetensor. Đây chính là base model mà stability AI phát triển. Về bản chất thì các checkpoint về sau do cộng đồng phát triển đều được tạo nên từ cơ sở dữ liệu gốc của model này.
- Bạn có thể phân biệt 2 thứ này theo kiểu mẹ => con. Nhưng thực ra là cũng chả cần phân biệt nếu bạn ko có ý định trở thành model creator. Bởi khi sinh ra đứa con thì cơ sở dữ liệu của model con bao gồm 1 phần kiến thức nền tảng từ mẹ và những kiến thức khác do creator muốn đứa con học. Về bản chất nó là tệp cơ sở dữ liệu dùng để truy vấn khi bạn gõ prompt vào. Thứ này gọi là Checkpoint.
- Các checkpoint sẽ lại trở thành base model khi model creator sử dụng checkpoint để tạo ra 1 checkpoint mới.



Testing

Base model/ checkpoint

- Hãy cẩn thận và giành thời gian xem các ảnh demo của 1 checkpoint kỹ càng trước khi lựa chọn. 1 Checkpoint quyết định output của bạn có tốt hay ko. Ví dụ bạn muốn vẽ chó mèo, nhưng lại lựa chọn 1 checkpoint mà cơ sở dữ liệu chỉ có gái (ví dụ chilloutMix), thế thì có thể nó vẫn vẽ ra con chó do checkpoint này được sinh ra từ base model mẹ có dữ liệu con chó, nhưng số lượng hình ảnh về chó đã mất đi rất nhiều để trống chỗ cho hình ảnh về gái. Điều ấy có nghĩa là ít dữ liệu thì có khả năng cao là ko vẽ ra hoặc sẽ vẽ ra 1 con chó dị dạng, hoặc sẽ ra 1 kiểu quái vật mình người đầu chó.
- 1 Checkpoint tốt giống như 1 đứa trẻ sau khi được học hành (trải qua quá trình training) có đủ nhận thức để hiểu những gì user muốn. Và cũng như những loài sinh vật khác thôi, đừng bắt 1 con cá biết bay, đừng cố bắt 1 model 2D anime cho ra 1 hình realistic, ultra realistic, photography realistic....



Biểu hiện của sự cố chấp vẽ 1 con rồng phương đông trên model ko có dữ liệu về rồng

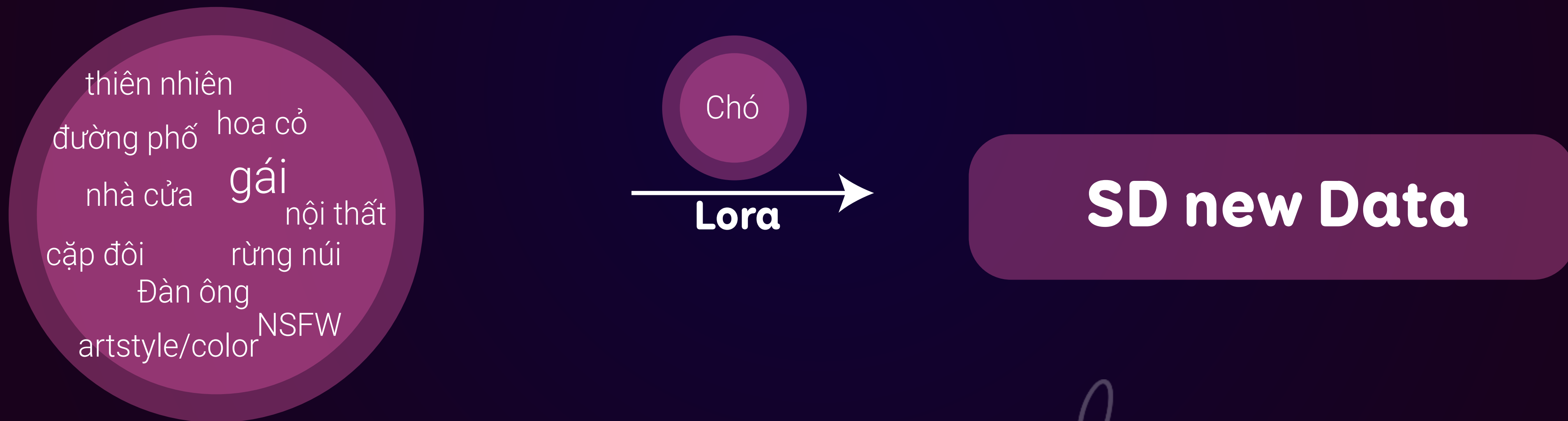
Texting

Lora/Lycoris/Loha

Lahin the futureland

Testing Lora/Lycoris/Loha

Có đến 3 hạng mục, nhưng đại loại thì cả 3 cái đều là 1 dạng mini checkpoint. Vì sao? Vì nó như phần bù khuyết cho các checkpoint. Ví dụ là bạn vẫn muốn vẽ con chó trên 1 cái checkpoint chỉ có gái? Hãy tìm 1 cái lora về chó. Lora có tệp cơ sở dữ liệu nhỏ hơn so với checkpoint rất nhiều lần, nhưng nó sẽ giúp bù đi phần thiếu của checkpoint, **định hướng** lại output. Cái gì mà checkpoint ko hiểu, ko có dữ liệu thì tìm 1 cái lora để bù đắp.



Checkpoint

Lahin the futureland

Testing Lora/Lycoris/Loha

Lahin the futureland

Ví dụ: Bạn thích nước ảnh, art style của checkpoint A, bạn muốn hình ảnh 1 phụ nữ mặc áo dài. Nhưng checkpoint ko có dữ liệu về áo dài, việc bạn cần làm là bổ sung dữ liệu cho checkpoint A bằng lora/locon... input của bạn sẽ được đi qua 1 lớp filter của lora vừa để lấy dữ liệu của lora và vừa định hướng lại kết quả. Bạn sẽ có kết quả đầu ra là 1 hình ảnh có chất liệu của checkpoint A cùng với trang phục của lora áo dài.

Checkpoint



+



=



Testing Lora/Lycoris/Loha

Khi sử dụng lora/lycoris/loha bạn sẽ tiếp xúc với 1 thứ gọi là trọng số (weight). Trọng số là mức độ ảnh hưởng của phần tử lên output. Ví dụ khi add lora áo dài vào 1 prompt bất kỳ nó sẽ có dạng: **<lora:aodai:1>**. 1 ở đây chính là trọng số của lora, tức là cơ sở dữ liệu của lora áo dài có 100% sức ảnh hưởng tác động lên kết quả cuối cùng. Và thường thì nếu chỉ tạo 1 hình ảnh chả có ai để weight = 1. Thường thì mình sẽ thấy lora hoạt động tốt trong khoảng 0.4-0.8. Bạn cũng có thể để thấp hơn nếu ko muốn cơ sở dữ liệu của lora tác động lên output quá nhiều mà chỉ muốn lấy dữ liệu của 1 số phần tử nằm trong lora.

Lahin the futureland



Trọng số lora quá cao sẽ dẫn đến khuôn mặt biến dạng, hình ảnh vỡ

Texting

Lora/Lycoris/Loha

Lahin the futureland

Trọng số này ko chỉ áp dụng cho lora/lyco... mà còn áp dụng cho bất cứ thứ gì trong prompt của bạn. Ví dụ bạn có prompt:

1 man with wings, flying, wearing blacksuit.

Nhưng kết quả vẫn cho ra 1 người đàn ông mặc áo đủ màu, bạn muốn nhấn mạnh cụm từ wearing blacksuit, để kết quả đầu ra chắc chắn rằng người này mặc đồ đen. Bạn có thể thêm trọng số như sau:

1 man with wings, flying, (wearing blacksuit:1.4).

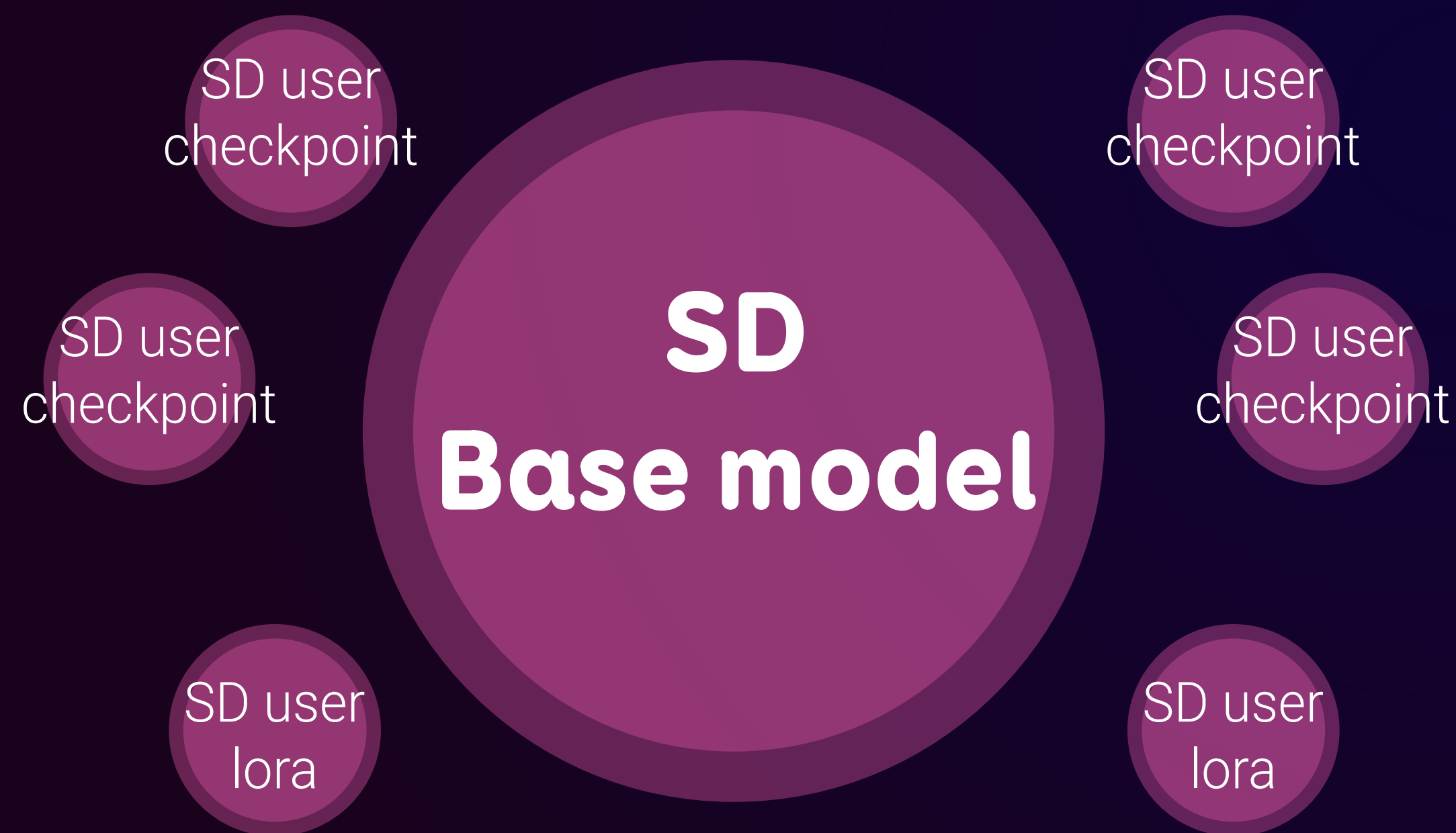
Điều này tương đương với việc cụm từ wearing blacksuit có 140% sức ảnh hưởng, so với việc bạn ko viết gì thì nó sẽ mặc định là 100%.

Có thể bạn sẽ thắc mắc tại sao lora thì phải điều chỉnh xuống dưới mức 100% còn các keyword khác thì phải trên 100%. Điều này hơi phức tạp để mà giải thích. Vì nếu viết ra thì chắc mất thêm vài trang nữa. Nên có thể mình sẽ đề cập sau (có thể là ở phần training model)

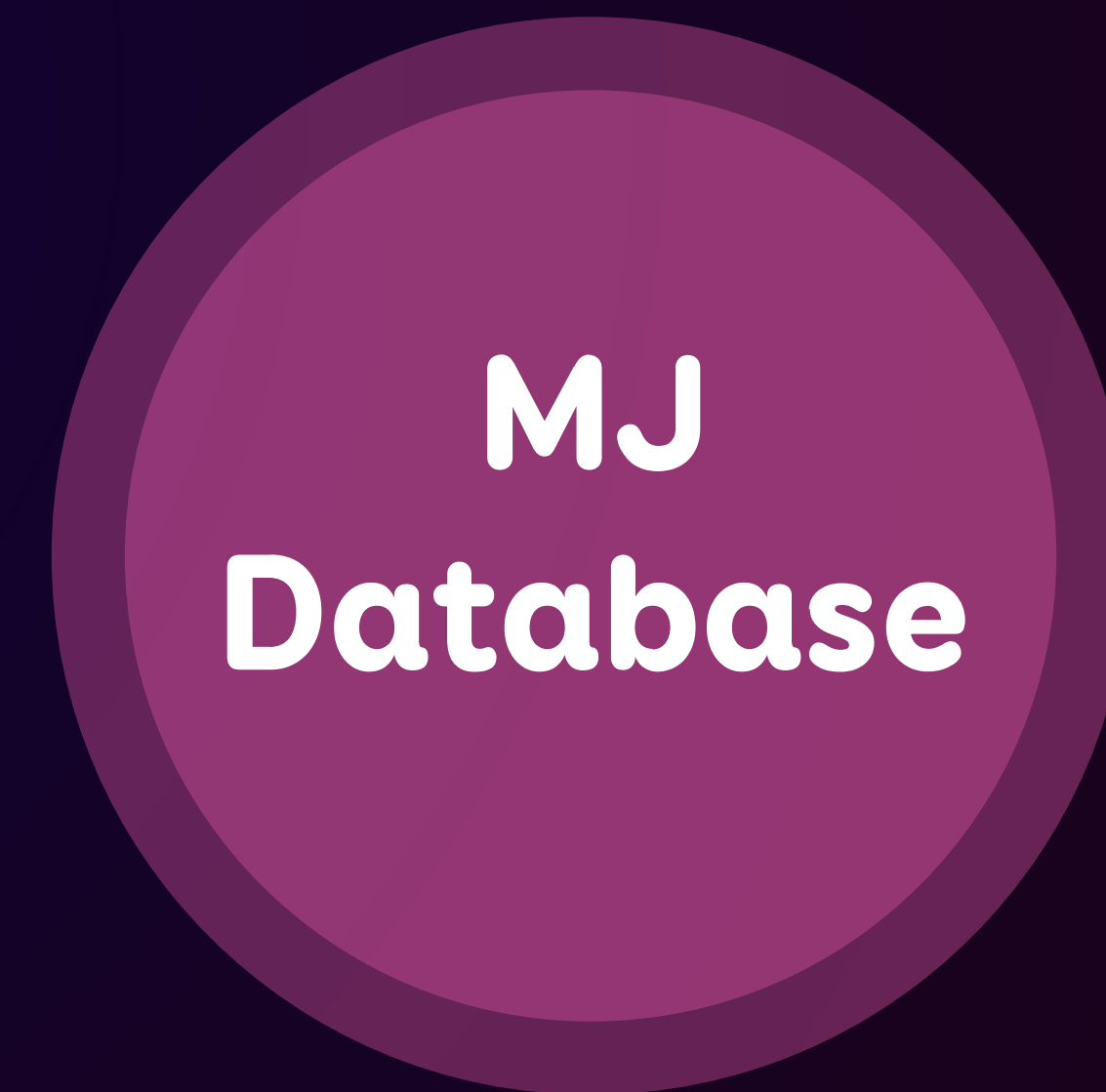
Testing Lora/Lycoris/Loha

Lahin the futureland

Đây là thứ khá cơ bản mà SD giúp user can thiệp nhiều hơn vào kết quả đầu ra. Ăn đứt MJ, bởi MJ ko cho user được tự lựa chọn Lora hay checkpoint, MJ chủ động lựa chọn cho user các lora/checkpoint phù hợp nhất. (cái này nó thông minh vãi chưởng vì đọc vị được user) Nhưng mà về bản chất MJ cũng sẽ trải qua bước lấy mẫu từ trong 1 cơ sở dữ liệu, và nếu thứ bạn đề cập tới ko có trong cơ sở dữ liệu của MJ thì nó cũng chả vẽ ra được. Lúc này thì SD giải quyết vấn đề dễ hơn nhiều.



Cơ sở dữ liệu của Stable Diffusion



Cơ sở dữ liệu của Mid Journey

Testing Lora/Lycoris/Loha

Chắc bạn sẽ lại thắc mắc là làm thế quái nào biết trong 1 lora có dữ liệu về cái gì? Dễ nhất là nhìn tên đoán ý. Lora Nhật Bình chắc chắn có dữ liệu áo Nhật Bình, Lora Hán phục chắc chắn có dữ liệu hán phục. Còn nếu muốn chi tiết hơn thì bạn có thể mở tab lora rồi chọn vào dấu (i) nó sẽ hiện ra 1 cái tab với 1 đồng chữ về dữ liệu train. Ở đây bạn sẽ tìm thấy thành phần cấu tạo nên lora hay cũng chính là cơ sở dữ liệu của lora.



```
{
  "ss_sd_model_name": "model.safetensors",
  "ss_clip_skip": "1",
  "ss_num_train_images": "2256",
  "ss_tag_frequency": {
    "China goddess": {
      "china goddess": 47,
      "lgirl": 47,
      "solo": 34,
      "dress": 7,
      "barefoot": 3,
      "black_hair": 17,
      "jewelry": 18,
      "long_hair": 12,
      "veil": 23,
      "braid": 1,
      "solo_focus": 1,
      "sitting": 1,
      "closed eyes": 1,
      "weapon": 7,
      "sword": 2,
      "white_dress": 2,
      "bow_(weapon)": 1,
      "head_out_of_frame": 1,
      "necklace": 1,
      "dancing": 1,
      "flower": 1,
      "phone": 1
    }
  },
  "ss_cache_latents": "True",
  "ss_caption_dropout_every_n_epochs": "0",
  "ss_caption_dropout_rate": "0.0",
  "ss_caption_tag_dropout_rate": "0.0",
  "ss_dataset_dirs": {
    "China goddess": {
      "n_repeats": 48,
      "img_count": 47
    }
  }
}
```

Lahin the futureland

Texting

Lora/Lycoris/Loha

Rồi bây giờ thì mình mới nói thêm về lycoris, loha, lora. Thật ra ban đầu SD ko có lycoris và loha mà chỉ có lora. 2 thứ kia về bản chất là 1 hình thái khác của lora. Nó ko có quá nhiều khác biệt cho người mới, nên bạn cũng chả cần cố gắng phân biệt làm gì. Cả 3 vẫn là 1 ổ cứng rồi cung cấp cơ sở dữ liệu bị thiếu cho checkpoint. Nếu bạn thực sự muốn phân biệt 3 thứ này thì thực ra là nó khác nhau ở mô hình đào tạo, mỗi thứ sẽ có 1 tính năng ưu việt và phù hợp cho từng tệp dữ liệu khác nhau. Mình vẫn phải để lại phần này và giải thích kỹ hơn ở những bài về training, bởi kiến thức này mình cho rằng ko phù hợp với bài 1.

Lora

Lycoris

Loha

Lahin the futureland

Testing

Base model/ checkpoint

Trang này là mình phải bổ sung thêm sau khi đã viết xong cả phần 1 rồi nhưng cảm thấy nhất định phải viết nếu ko thì sẽ rất là lẩn cẩn.

Như ở đầu mình có nhắc về việc Lora/lyco... có tính **định hướng** output. Ở đây tức là khi sử dụng lora tùy thuộc vào trọng số của bạn để cao hay thấp sẽ dẫn đến việc các sampler sử dụng nhiều hay ít dữ liệu trong ổ cứng rời này. Nếu trọng số của lora quá cao thì có thể đám nhân viên sampler của bạn sẽ bỏ qua dataset của checkpoint luôn, mà chỉ sử dụng dataset của lora.

Nếu bạn sử dụng 1 lora về trang phục, nhưng bằng 1 cách nào đó kiểu tóc của nhân vật trong output không cách nào thay đổi, dù bạn sửa prompt như thế nào đi chăng nữa. Có nghĩa là do người train lora đó đã vô tình khiến AI tưởng rằng kiểu tóc đó là 1 phần tử bắt buộc khi sử dụng lora. (Chính mình là thằng đã train ra mấy cái lora đàn độn như vậy khi thiếu hiểu biết =))))). Đây hoàn toàn là do lỗi kỹ thuật của creator tuy nhiên nếu bạn muốn giải quyết thì hãy giảm trọng số của lora xuống để các sampler cân đối giữa việc sử dụng dữ liệu từ dataset của cả lora và checkpoint.

Lahinthefutureland

Texting Lora/Lycoris/Loha

Nếu bạn là 1 generator thì đây chính là phần sáng tạo nhất

Kết hợp các lora sẽ cho bạn những kết quả mà có thể bạn ko tưởng tượng ra. Ví dụ mình có lora áo Nhật Bình, nhưng dữ liệu của trang phục Nhật Bình khi train thì là ảnh mẫu đứng 1 chỗ cười tươi để chụp ảnh. Giờ mình muốn nhân vật mặc Nhật Bình ngẫu lòi, khí thế 1 chút. Mình sẽ tìm 1 lora về attack pose và sử dụng cả 2 lora trong 1 prompt. Kết quả bạn có thể thấy như hình.

Tất nhiên là khi mix các vị vào với nhau sẽ ko còn là vị nguyên bản, nhiều thứ bị bóp méo cũng như thay đổi. Bây giờ công việc của bạn là phải thử tư duy và đặt trọng số cho các lora trong cùng 1 prompt như nào cho hợp lý. Tất nhiên vẫn theo nguyên tắc là mình muốn lora nào tác động nhiều thì để trọng số cao và ngược lại.

Mình từng đọc đc 1 bài viết nói rằng tổng trọng số của lora phải bằng 1. Ví dụ dùng 3 lora a-b-c thì **$weight\ a + weight\ b + weight\ c = 1$**

Điều này theo mình là ko đúng. Bạn thích đặt bao nhiêu là tùy bạn, và nó chả ảnh hưởng quái gì. Chỉ là hãy chú ý rằng khi mix càng nhiều thì càng khó kiểm soát, đặc tính ban đầu của các lora cũng mất đi do bị pha trộn.



Texting

Something other

Lahinthe futureland

Texting

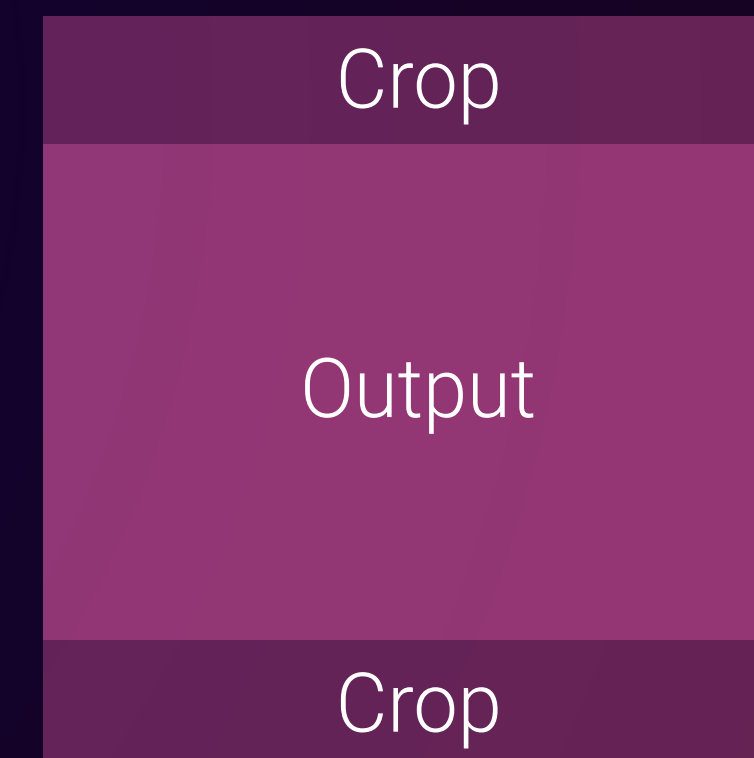
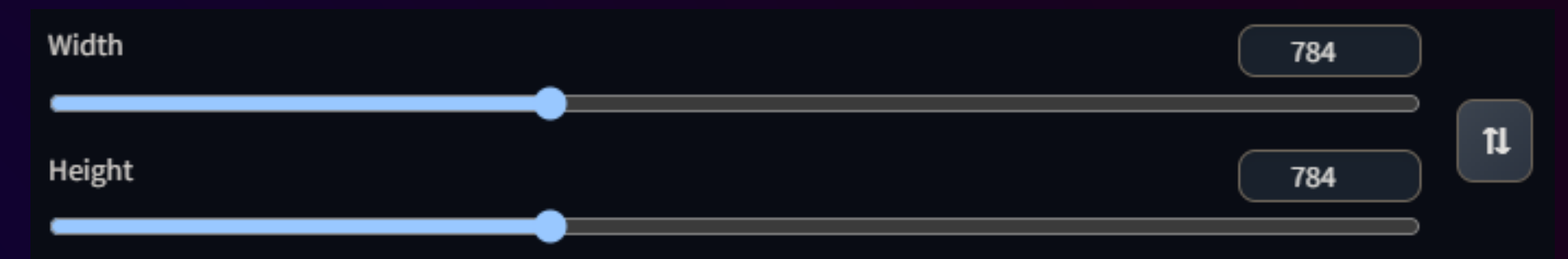
Something other

Width & Height: Rộng - dài. Tóm lại nó là kích thước ảnh mà bạn muốn tạo.

Do dữ liệu khi đào tạo các mô hình AI đều là ở dạng hình vuông (512x512) cho nên thực tế thì kết quả đầu ra tốt nhất cũng ở dạng vuông.

Trong trường hợp bạn muốn kích thước hình ảnh là 1 ảnh ngang, hoặc dọc ví dụ là 1920x1080 thì về bản chất SD sẽ tạo ra 1 hình ảnh có kích thước 1920x1920. Sau đó crop về 1920x1080. Cho nên khá nhiều trường hợp ảnh bị crop mất chân, tay đầu.... nhưng nó cũng chả phải vấn đề gì lớn, trừ khi bạn đặt ra cái kích thước quá vô lý kiểu như tỉ lệ 1:10

Lahin the futureland



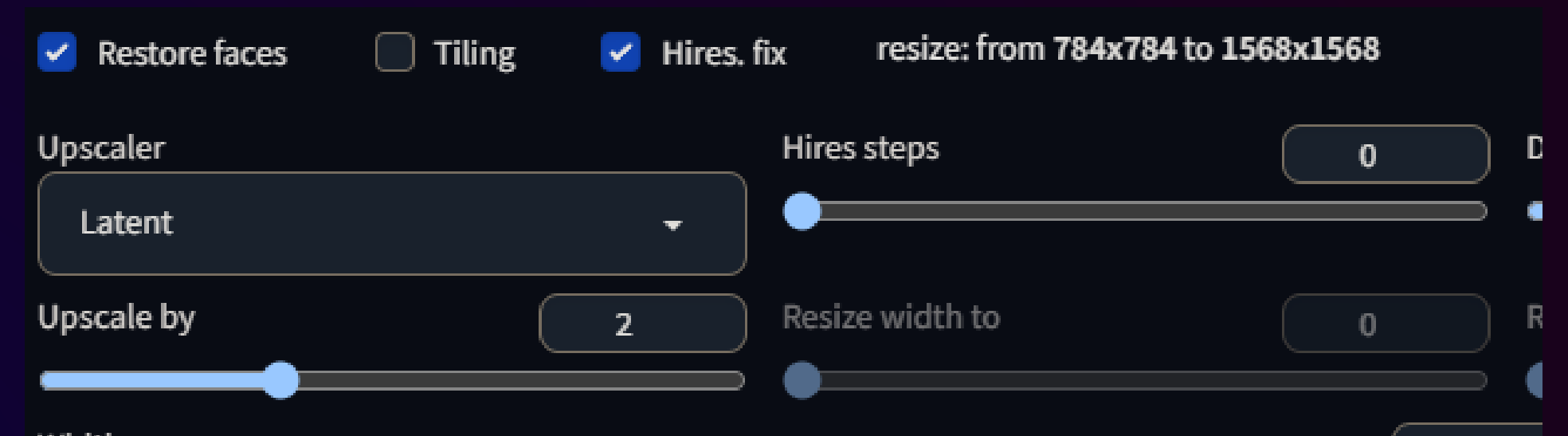
Texting Something other

Hires fix: Nói về nguyên lý thì thằng này là 1 công đoạn hậu kỳ ảnh. Sau khi process chạy xong và tạo ra 1 hình ảnh với kích thước bạn đã setup ban đầu thì Hires fix sẽ bắt đầu hoạt động.

Cơ chế là nó sẽ phóng to hình ảnh được tạo ra theo mong muốn của bạn (tùy bạn muốn up lên mấy lần, và VGA của bạn có gồng nổi ko) và add thêm detail, phương pháp add thì tùy thuộc vào model upscale bạn lựa chọn khi up. Và có vẻ như nó có khả năng fix thêm 1 vài lỗi lặt vặt khi hình ảnh ở độ phân giải thấp quá nhỏ để fix (cái này mình ko chắc chắn lắm)

Cũng tương tự như sampling method, thì ở công đoạn này bạn cũng có mấy ông nhân viên làm nhiệm vụ upscale hình ảnh, và mỗi ông cũng có workflow khác nhau dẫn đến những kết quả khác nhau. Cái này các bạn chủ động tự tìm hiểu nhé. Mình thì thường dùng 4x ultrasharp và có vẻ nó hoạt động tốt trong hầu hết các trường hợp.

Lahin the futureland



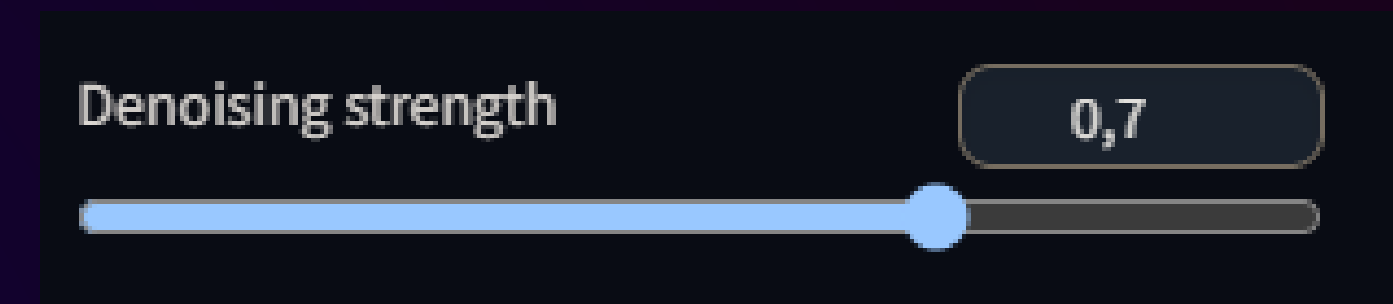
Hires steps: cái này tương tự sampling step. Nếu hires step quá thấp thì nhân viên của bạn làm việc cầu thả, ko phải add detail mà là vẽ nguệch ngoạc lên 1 cái ảnh đang tử tế. Nếu muốn 1 con số thì nhiều người khuyên dùng từ 15 đổ lên.

Hires steps = 0 thì số step sẽ bằng với sampling steps. Mình luôn đặt ở mức này. để khỏi mang tiếng thiên vị các bộ phận trong cùng 1 doanh nghiệp =))

Texting

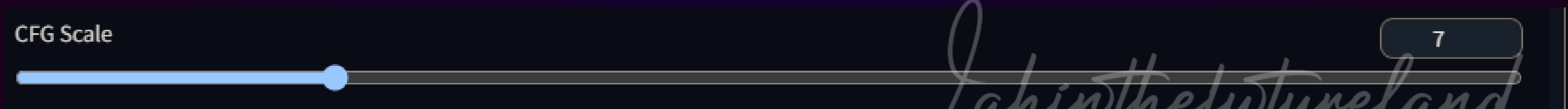
Something other

Denoising Strength: độ khử nhiễu trong Hires fix. Như đã giải thích ở trên thì quá trình Hires fix AI sẽ vẽ thêm detail lên output. Và nó vẽ gì thì mình phải kiểm soát. Nếu đặt chỉ số Denoising Strength này quá thấp nó sẽ ko làm gì cả (tác động quá ít ko có tác dụng) còn nếu đặt quá cao thì cơ bản là nó sẽ vẽ mẹ nó lại cái ảnh khác luôn. Cái này thì khá nhiều cách dùng, nhưng với text2img thì mình hay đặt trong khoảng 0.35-0.5



CFG Scale: Vùng an toàn mà bạn đặt ra cho nhân viên.

Con số này càng thấp thì vùng an toàn càng nhỏ, điều ấy đồng nghĩa với việc bạn sẽ gò bó nhân viên mình trong quá trình sản xuất. CFG Scale càng gần mức 1 thì output càng sát với prompt, nhân viên của bạn sẽ chỉ đâu đánh đấy, còn CFG Scale cao lên, vùng an toàn được mở rộng thì tính sáng tạo càng cao nhưng cũng đồng nghĩa là nó sẽ khác so với yêu cầu ban đầu nhiều hơn. Mình thường để cái này ở mức mặc định là 7, hoặc nếu muốn tăng giảm thì khoảng từ 5-12 là hợp lý. Thấp quá sẽ cho ra hình ảnh tẻ nhạt, thiếu màu sắc, còn cao quá thì nhân viên của bạn ra khỏi vũ trụ, sáng tạo ra những thứ mà con người ko thể hiểu được đâu.



Lahin the futureland

Texting

Workflow

Lahin the futureland

Text2img Workflow



Mỗi người sẽ có 1 cách làm riêng, và điều này ko áp dụng cho tất cả mọi đối tượng. Tuy nhiên ở đây mình xin phép chia sẻ workflow của mình khi làm việc với text2img.

1. Xác định rõ thứ mình muốn vẽ: Chủ thể là gì? (bắt buộc) - tư thế, góc nhìn...
2. Xác định tiếp các bối cảnh, phong cách, màu sắc... bước này có thể có hoặc ko trong trường hợp bạn ko nghĩ ra
3. Lựa chọn checkpoint phù hợp với phong cách bạn muốn cũng như có dữ liệu của chủ thể.
4. Sắp xếp prompt theo công thức **Quality + subject + accessory + environment + artstyle**
5. Negative prompt
6. Lựa chọn lora nếu muốn và nhớ đặt trọng số cho lora ở mức vừa phải
7. Tinh chỉnh các phần sampling method, step, kích thước
8. Nhấn nút generate và thử sai! Sau khi nhận kết quả đầu tiên hãy nghĩ xem output này đang thiếu gì mình muốn? và thừa thứ gì để chỉnh sửa prompt cũng như negative prompt. Không có công thức hay thông số đúng cho mọi trường hợp. Hãy cứ generate và tìm ra thông số tốt nhất cho bản thân.

Trong 1 số trường hợp người ta còn dùng controlnet trong text2img và 1 số extension khác. Tuy nhiên mình chưa muốn đề cập tới trong phần này, chủ yếu là vì mình lười quá rồi, vì thế các bạn có thể chờ phần sau nhé.

Ở cuối bài mình có đặt 1 số keywords cho việc điều chỉnh góc nhìn của nhân vật trong trường hợp các bạn muốn vẽ người và 1 sơ đồ về cấu trúc vận hành của SD (cái này hơi khó nếu bạn ko biết gì về IT). Hy vọng những thứ này có thể giúp các bạn dễ dàng tiếp cận hơn trong quá trình sử dụng Stable Diffusion.

